



E.J OURSO COLLEGE OF BUSINESS
SDEIS DEPARTMENT
Nicholson Extension
Baton Rouge, LA 70803
USA

GABRIELE PICCOLI, PH.D.
2219 Business Education Complex
Tel: +1 (225) 578-8184
gpiccoli@lsu.edu

ISDS 3100: Information Systems Foundations

Fall 2026

AI Lab: Vibe Coding and Agent Skill development

The goal of this semester project, to be completed with a pair-programming approach, is to practice what we learn in class. Specifically, the project focuses on the Generative AI portion of the course. We will practice vibe coding, and we will learn about (and create) agent skills. If you are wondering why, as a future manager, you should be learning to vibe code and to create agent skills, here is some food for thought. You may never write software for a living, but you will increasingly *direct* systems that do work for you. Vibe coding and building agent skills are small, concrete examples of the core managerial move in the age of GenAI. You must be able to clearly specify what a capable system should do, encode that specification so it runs consistently, and judge whether the output is achieving the expected levels of performance. You are the navigator. The AI can produce the code or automate the work, but you must decide what *performance* and *results* must be. That judgment is the part that doesn't get automated. The more you practice, the better you become at it. These are frontier skills for a manager. Now is your time to develop them.

General requirements and expectations

The two separate projects described here are independent. For each one you will be working in teams of two, but you will be able to change your partner once the vibe coding project is complete. It is expected that you will act as a professional, reliable partner to your teammates. If your teammate on the vibe coding project does not share your same commitment to quality work, professionalism or learning, you should select a new teammate for the second project.

Pair programming

In this project you choose a collaborator and you work as paired programmers. Pair programming is an agile software development technique (see Chapter 10 of your book), where two programmers work together at a single workstation to complete a coding task (e.g., to vibe code a website). One of the two developers takes the *driver role* and writes the code. The other one, taking the *navigator role*, reviews the code as it is being typed and thinks at a higher level about the project spotting the potential for bugs, architectural challenges, new features. In a vibe coding environment, the driver is not actually writing the code but instead prompting the generative AI system that writes the code.

Website design and GitHub hosting

The first project consists in creating a personal website, one for each team member. You will take turns, working on one website at a time in the pair programming modality. Note, while you will be pair programming

each website, grading will be individual – each developer is graded on their own personal website (see grading section). You shall not use any website builder (e.g., Wix, Squarespace) or backend providers (e.g., Lovable, Supabase). Rather you will create the website on your computer using Google's Antigravity IDE, test it, and then upload the code to GitHub (make a free account if you don't have one yet).

The following are the minimum requirements for each of your websites:

- Clear design: You must choose a *hybrid layout* for your design. Your high-level content (about, skills, contact) must be in a single-page design. You must have at least one separate page to house your resume and a second separate page to describe one project that highlights your abilities or passions.
- Content: Your website should open on your single-page and provide brief descriptions of you (profile), your skills (competencies), your experience (jobs held) and your contact. Your Resume and Project(s) page will expand on those highlights.
- Design elements: The visuals, graphic elements, navigation appearance and text must work together to quickly communicate your purpose and orient users to your competencies and experiences. Your website must have a consistent page design, be professional and targeted to potential recruiters (your main audience).
- Hosting: Your code must be on your computer and be pushed to GitHub in your own GitHub account. Set the repository to be public for inspection.
- Your website should be hosted and publicly reachable in one of the following two free hosting services: GitHub Pages or Cloudflare Pages (both free). If you so choose, you can purchase a URL and point it to your website, but this is not a requirement.

Agent skill design and deployment

The second project consists in creating one agent skill for Claude Code. Working with your teammate, you will ideate, design, build, test, and deploy a single working skill. As noted in the general requirements, you may keep your partner from the vibe coding project or select a new one for this project. The team produces one artifact and shares one grade.

An agent skill packages a repeatable task so that an AI agent can perform it reliably and consistently, without being re-taught each time (see Agent Skill Primer). So, this is an exercise in automating expertise with Generative AI. Building a skill is a design task as much as a coding task and minimal coding ability is required. However, it pays to have a clear understanding of computing concepts and you, as an IS professional, are best positioned to become a great agent skill writer.

Requirements

You have free choice of topic. Your skill must be genuinely useful and appropriately scoped. It does not have to be a business or work skill. But it must help with a task you (or someone else) would normally have to repeatedly perform manually. A solid skill for this project is: a) repeatable: it automates a task someone would do more than once, not a one-time request. Formatting *any meeting transcript* into structured minutes is a good example. Summarizing *one specific article* is a bad example; b) useful to a defined user: it must be able to name a specific person (myself) or role (the secretary of AITP) who benefits from the skill. You design for that person; c) appropriately scoped: the skill does one thing well. A focused skill that reliably does a single job beats an ambitious one that half-does five. When in doubt, narrow it. Formatting *any meeting transcript* into structured minutes - good example. Being an *office assistant* to help with secretarial tasks - bad example.

Deliverable

Your deliverable is a working agent skill, installed and tested in Claude Code, that meets the following minimum requirements:

- A *well-formed SKILL.md* with valid frontmatter — a clear name and, most importantly, a precise description that tells Claude *when* to use the skill.
- *Structured instructions* in the body that lay out, step by step, how the task should be performed. You must test the skill and ensure it performs consistently and doesn't depend on the user prompt.
- *One or more supporting resources* bundled with the skill (for example, an example output, a template, a reference file, or a short script).
- *Demonstrated reliability* so that when we run the skill you provide it triggers correctly and produces consistent, correct results across at least two separate test runs with different inputs.

Suggested workflow

The following is a suggested progression in the creation of the skill.

- Ideate with your teammate and write one sentence describing exactly what your skill does and for whom.
- Create the skill folder and SKILL.md (see Agent Skill Primer).
- Draft the triggering description, then the step-by-step instructions.
- Add your supporting resource (template, example, or script).
- Install it, then test it by triggering it with real inputs. Watch what breaks.
- Iterate, tighten the description and instructions until it works reliably across at least two different inputs.

Calendar

All work is due on the day of the deadline at 11:59:59 PM unless otherwise communicated. There is a 30% penalty per day of delayed submission on all work. The updated list of key dates and deadlines for the semester is available on Moodle.

Grading and evaluation

See syllabus for grading percentage of each element of the semester project. For the vibe coding project each person is assigned the grade relative to their own website (i.e., grading is individual). On the agent skills project the team produces one artifact (i.e., one agent skill) and shares the same grade. We assume equal contribution to the final deliverable. It is your responsibility to reach out to the instructor or GA early in the project if you believe that your teammate is not contributing adequately to the effort.

Vibe Coding Primer

This handout helps you get started on your project using Google Antigravity as your IDE. No prior coding, GitHub, or command-line experience is assumed. The best approach would be to practice this with the classic “Hello World” page so that when you have your vibe coded website built you can move efficiently.

We will cover:

1. Set up your tools and accounts (one time)
2. Build and test your site on your own computer
3. Upload to GitHub
4. Preview the website locally, iterate and push major improvements

Note: You must be able to *see and edit the actual code* (the `.html`, `.css`, `.js` files) in Antigravity IDE’s editor. No Wix, Squarespace, Lovable, v0, or any tool that hides the code from you. Antigravity’s agent *writes* the code, but the actual files it creates you can inspect, edit, and own (that’s the point of vibe coding).

Set up your tools and accounts (do this once)

1. Google Antigravity IDE (free). Download the installer from the official page at antigravity.google and run it. Note that there are multiple products called Antigravity and you need to get the Antigravity IDE.

Launch the installer:

- When prompted, choose *Start fresh*.
- Pick a theme.
- On the Agent Manager / mode screen, choose *Review-driven development* (recommended). You stay in control and approve actions, while the agent helps. This is the approach we want for learning.
- Sign in with a *Google account* (create a free one if you don’t have one or don’t want to use an existing one) and select a model (*Gemini 3 Pro* is the default; Claude and others may also be listed and may give you limited usage. You can experiment with various models later).

Antigravity is built on a familiar VS Code–style editor, so the layout will feel standard: a file editor in the middle, an Agent Manager panel where you give the AI instructions, and a built-in browser preview for testing.

2. Git (free). Git is an open-source distributed version control tool. It is used by software developers to track changes in files and coordinate work among multiple people. It is the software you will use to *push* your code to GitHub. Before the installation check if you already have it. For example, on a Mac you can check in the Terminal app by typing `git --version`. If Git isn’t installed, your Mac will offer to install it. If needed, you can download Git from git-scm.com and install it by following the prompts.
3. GitHub account (free). Sign up at github.com. Pick a professional username, if you don’t have an account yet, this is your opportunity to create one that recruiters will see. Be smart and be professional. `jane-smith` – professional. `Xxgamergirlxx` – not professional.

Evaluate progress by launching Antigravity IDE, typing `git --version` and verifying that Git is properly installed. Visit your new GitHub account and verify that it is created and you can access it with your credentials. Create the standard “hello world” project to make sure everything works correctly

(<https://docs.github.com/en/get-started/using-github/hello-world>). Once you clear this checkpoint you are in business and ready to work on your personal website.

Build and test your site on your own computer

The following requirements will help you keep the structure of your project clean and your work simple and efficient. First, create a new folder in a specific location (e.g., the folder with your class documents). Give it a clear name (e.g., `firstname-website`). Learn and apply good naming conventions (e.g., no spaces, all lowercase, dash separators). Second, initialize the main files in your website. You can create them yourself or prompt the agent appropriately to create them. Start with the following files:

- `index.html` — this is your home/landing page (the single-page part: Profile, Skills, Experience, Contact). Browsers and GitHub Pages look for `index.html` first.
- `resume.html` — your resume page.
- `project.html` — your project page.
- `styles.css` — your shared styling, so every page looks consistent.

To vibe code with the agent, make sure the Agent Manager panel is open, make sure your folder is selected, choose a model, and simply describe what you want in plain English. For example: *“I want to create my personal website. It has a very specific structure. Create the following four files to initialize our project: `index.html`, `resume.html`, `project.html` and `styles.css`.”*

To verify the work of the agent, you should see files in the Explorer side panel (on the left). Open each one and browse the code. If you know the language used (e.g., HTML) you will recognize what is going on. If you don't know the language, it's ok. For this project you are not expected to directly modify any code (but you of course can if you so choose). The agent writes the code into your files. The key learning point here is that, by having direct access to the code and being able to inspect it, you directly own all the elements of your final product.

Once you are comfortable with the workflow and the tools you can iterate by prompting the agent to make changes and improve your website as you see fit with follow-up instructions (e.g., “use a navy and white palette,” “the contact link is broken, please verify what is wrong”).

A great feature of GenAI agents is that you can ask them to explain what they are doing and what is going on. This is perhaps the most valuable part of this project, and you should take advantage of it! As you interact with the agent and ask questions, you are improving your ability to architect systems and code.

Make sure to not accept anything you don't understand. Simply ask the agent to explain why a task is needed, why something is not working, whether there are better designs available, or what it suggests doing in specific situations. You will be amazed by how much you learn through these “coaching” sessions. As you get better and more comfortable with the agent you can dive deeper into more complex tasks.

Upload to GitHub

GitHub will serve not only as the host of your final product, but also as a remote backup of your work. So, you should not wait to complete your project before you push it to the repo. You should instead do so as you go along, at the end of each day of coding or when you complete major milestones. We suggest that you do your first commit and push once you have the bare bone skeleton of your web site.

First, you will create the repository (repo as developers call it) on GitHub. There are several ways to do so: <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository> You are probably best off doing it on the GitHub website directly. You will choose a repo name (make it meaningful). You must set the repo to *public* to later publish as a GitHub page. Make sure to leave the README file empty for this repo, you will add it later. Second, with the repo created, you can push your code. You can do so from the Terminal app or directly from the Antigravity IDE. The first time you will need to sign in with your credentials. The sign in process evolves over time, so review the requirements before your first commit. You will know that you have successfully completed this step when refreshing the repo page on GitHub will show you the updated code, the correct timestamp, and the latest commit message.

Preview the website locally, iterate and push major improvements

At various stages of development, you can ask the agent to preview the work (e.g. *“Start a local preview of this site and open it in the browser”*). Antigravity IDE serves your site in its built-in browser preview. Every time the code changes, refresh the preview to see it. This is your iteration workflow: change → inspect → refine. As you get comfortable with the vibe coding iteration process, ask your agent how to start a local server so you can preview the website in your browser (rather than the built-in browser of Antigravity IDE). Sometimes websites look different on different browsers (e.g., Chrome, Safari), so previewing it in multiple browsers is a good website design practice.

Once you like the results in terms of content and design, here is a checklist:

- Every navigation link works.
- The design looks consistent across all three pages.
- The content reads well and looks professional to a recruiter.
- Resize the preview narrow — does it still look OK on a phone-width screen?

At this point you are mainly testing locally; nothing is on the public internet yet. But once you make major improvements you should push them to your GitHub repo. Follow the sequence of Git commands:

- `git add .`
- `git commit -m "descriptive commit message"`
- `git push`

Note that you can push code from the terminal or from Antigravity IDE. You will likely identify your preferred method and stick with it. Remember, Git is a version control system that allows you to revert to previous commits if you make mistakes or don't like a set of improvements. GitHub is your remote backup, it ensures you don't lose your work accidentally. Use these tools to vibe code like a pro.

Publish your personal website live with GitHub Pages

Your code is on GitHub, but not yet visitable as a public website. To publish the work for free as GitHub Pages you follow these steps:

- In your repository, click Settings (top menu) → Pages (left sidebar).
- Under Branch, choose main, keep the folder as / (root), and click Save.

Wait a few minutes, then refresh the Pages settings screen. A green box appears with your live address:

<https://YOURUSERNAME.github.io/firstname-website/>

This is your public website, the one you will share with recruiters, family and friends. You can see why choosing a professional and descriptive repo name is important. Test every page and link on the live version of your website using the public URL. Be sure to test on multiple browsers and multiple devices – PC, tablet, smartphone. Your website should be responsive and display well on each platform. You will likely have to get back into Antigravity IDE and iterate further. Pushing the new code to GitHub will automatically update the public website.

Agent Skill Primer

An agent skill is a packaged set of instructions that “teaches”¹ an AI agent how to do a specific, repeatable task well, so it can do that task reliably every time, without being re-taught. Think of it as an instruction card you write once and hand to a very capable but literal assistant – one with limited ability to judge and fill in the blanks of what you want. The power of a skill, versus a one-shot prompt, is that instead of re-explaining “here’s how I like meeting minutes formatted” every single time, you create the skill once, and the agent reaches for it whenever minutes are needed. This is the defining characteristic of an agent skill. Whereas a prompt is a one-time instruction that works for that moment, given the current state of the context window, a skill differs in three important ways:

- *Reusable*. You write the expertise once; it’s always available to the agent.
- *Self-triggering*. The agent recognizes when the skill applies and uses it on its own.
- *Packaged*. It can carry supporting materials with it so the agent has everything it needs.

In short: a prompt is *asking* the AI agent to do something once in a chat. A skill is *equipping* the AI agent with the capability to do something repeatedly and permanently.

Technically, a skill is simply instantiated as a *folder* on your computer. Inside it, one file does the essential work, and everything else is optional support. The key element is the required SKILL.md file.² This is the heart of the skill — a plain text file with two parts: a) the *frontmatter*: a few lines at the very top, fenced by a three-dash marker (i.e., “---”), that carry a *name* and a *description*. The description is the most important thing you write, because it tells the agent when to use the skill. A vague description means the skill never triggers, or triggers at the wrong time. A precise one makes it fire exactly when it should; b) the *body*: written in plain Markdown below the frontmatter it is the step-by-step explanation of how the agent should complete the tasks and the instructions it must follow. This is written in English, not using code, as if you were talking to a smart new collaborator. A clear, ordered, and specific explanation of what a good result looks like is key. Supporting files are optional for the skill, but you are expected to use them in this project (see project description). Supporting files are a powerful addition to the SKILL.md file because they give the agent access to extra materials. They are usually organized into folders called *references/* (documents the agent can consult), *assets/* (templates or example outputs), and *scripts/* (small programs the agent can run). These keep the main instructions focused while giving the skill the instruments it needs to do the job well.

Skills in Claude Code live as folders on your filesystem, in one of two places:

- *Personal*. These skills are stored in ~/.claude/skills/ and are available to you across all your projects.
- *Project-specific*. These skills are stored in .claude/skills/ inside a particular project folder and will be bundled with that project.

¹ You should resist anthropomorphizing Generative AI systems and models, using terms like “think,” “understand,” and “teach.” Quoting the terms is a simple way to remind the reader (and yourself) that a computer system is not human. For simplicity we will not quote anthropomorphized terminology here.

² As you enter the world of agent skills, and agentic AI in general, you need to become familiar with the *.md file type. An .md file uses Markdown syntax to create formatted text in a simple, plain-text format. It relies on easy symbols and is highly readable to humans while remaining lightweight and easy for GenAI models to ingest and parse. Markdown files are best viewed in an editor, such as *Sublime Text*, rather than word processors such as MS Word.

Once a skill is in place, there are two ways it gets used:

- *Automatically*. Claude reads the description of each available skill and, when your request matches, uses the relevant skill on its own. This is why the description matters so much, it's an automatic trigger.
- *On demand*. You can invoke a skill directly by typing / followed by its name. This way the skill is "force called" into action.

You don't upload anything to a server or fill out a form. A skill is just files in a folder that the agent knows how to read. The quality of the agent skill you develop depends on the three characteristics described in the project requirements. The skill must be a) repeatable; b) useful to a defined user; c) appropriately scoped.

To learn more about agent skills you can visit:

- Anthropic's own description: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- The Agentic Loop newsletter: <https://www.pulsemcp.com/newsletter>